

AI Course

Capstone Project Final Report

For students (instructor review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Bayan: On-Device Arabic Text Simplification for Readers with Dyslexia

تبسيط · Simplify

23/09/26

Cogni

Marwan Elamami
Abdulrahman Khengari
Ahmed Alaeb
Sanad Ali
Mohammed Thabet
Abdul Majid Mraied

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Members and Role Assignments
- 1.4. Schedule and Milestones

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Design

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. User Interface
- 3.5. Testing and Improvements

4. Projected Impact

- 4.1. Accomplishments and Benefits
- 4.2. Future Improvements
- References

5. Team Member Review and Comment

6. Instructor Review and Comment

1. Introduction

1.1. Background Information

Dyslexia impairs reading through phonological processing, and Modern Standard Arabic adds three obstacles that tools built for Latin scripts do not address. Short vowels are normally left unwritten, so one written form carries several readings: كُتِبَ is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books), and the reader must resolve it from context. Clitics attach directly to the stem, producing long words that resist chunking. Prose chains clauses with و and ف across whole paragraphs, so one sentence can exceed working memory.

Adding every diacritic is not a complete answer; it adds visual clutter of its own. The intervention we pursue is to *simplify first, then vocalise*. When the project started we found no deployed Arabic dyslexia-support tool and no large, openly reusable Arabic simplification corpus. That shaped both what we built and how much of our effort went into data and evaluation.

1.2. Motivation and Objective

Readers meet difficult Arabic inside other applications — a message, a news page, a PDF — not in a reading app. Bayan therefore appears where the text already is: selecting text on an Android phone offers تبسيط (*Simplify*) in the system text-selection menu, and the simplified text opens in a sheet over the same app. Every model runs on the phone, so the reader's text never leaves it and the tool works offline.

We committed to four measurable objectives. Table 10 in subsection 4.1 reports where each one stands today.

- 01** A simplifier that beats *copying the input* on the sentences a human actually simplified, on held-out test data it never saw.
- 02** Diacritics restored on the simplified text without altering its letters.
- 03** The whole pipeline running on a mid-range Android phone, within 250 MB for the simplifier.
- 04** Evidence from readers: a human evaluation with Arabic readers who have dyslexia, with informed consent.

1.3. Members and Role Assignments

Every member has merged work into the team repository; the evidence column cites the pull requests (#) and issues that hold it.

1.4. Schedule and Milestones

The project runs from 13 September to the pitch on 6 October 2026. At the time of writing (23 September, day 10 of 23) the first build-and-measure cycle is complete: a model has been trained, scored against a baseline on locked test sets, and diagnosed, and the application shell exists. Figure 1 shows what was done, what is running, and what remains; Table 2 compares the dates we set with the dates we met.

Table 1. Members, roles, and the work each has delivered so far.

Member	Role	Delivered (evidence)
Marwan Elamami	Team lead; integration; on-device model	Product-form decision (#9); evaluation protocol and fixes (#16, #18); model 1 diagnosis; Baseet acquisition and leakage filtering; Action Plan; this report
Abdulrahman Khengari	Data lead	Validators for generated pairs (#10, #11); CAMEL readability check (#17, #20); readability classifier; synthetic corpus in progress (#8, #12)
Ahmed Alaeb	Model training	SAMER conversion; model 1 trained and documented (#5, #22)
Sanad Ali	Diacritization; co-training	Four-system diacritization benchmark; Libtashkeel selected (#6, #23)
Mohammed Thabet	Android application	PROCESS_TEXT plugin, result sheet, 41 tests (#14, #21)
Abdul Majid Mraied	Evaluation	Scoring and leakage scripts (#7, #15); locked-set converter, copy baselines, generate-and-score path (#19, #24, #26)

Table 2. Milestones: target against actual.

Milestone	Target	Actual	Status
Evaluation scripts ready (#7)	Mon 21 Sep	Mon 21 Sep	Done
Diacritizer selected (#6)	Mon 21 Sep	Mon 21 Sep	Done
Android plugin shell (#14)	—	Mon 21 Sep	Done
Action Plan submitted	Tue 22 Sep	Tue 22 Sep	Done
Model 1 scored on the locked test sets	Wed 23 Sep	Wed 23 Sep	Done
Synthetic corpus exported (#8)	Fri 25 Sep	—	In progress
Model 2 trained and scored	Tue 29 Sep	—	In progress
Model running on a phone, size and latency measured	Fri 2 Oct	—	Not started
Human evaluation with readers	Sat 3 Oct	—	Not started
Final submission / pitch	5 / 6 Oct	—	Not started

2. Project Execution

2.1. Data Acquisition

We acquired five corpora (Table 3). SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed, and no SAMER sentence is reproduced in this report. Baseet is used with its authors' permission. Two test splits — SAMER test and BAREC test — were locked before any training and are never trained on.

A sixth source, simplification pairs generated by a large language model and admitted only through validators and human review, is being produced (#8, #12). The validators are merged (#11, #20); the corpus has not yet been delivered, and nothing in this report was trained on it.

2.2. Training Methodology

Candidate base models. Simplification is a sequence-to-sequence task, so every candidate is a pretrained encoder–decoder transformer: an encoder reads the whole source sentence, and a decoder writes the simplification one token at a time while attending to it. We short-listed three Arabic models of this kind (Table 4). Architecture figures are read from each model's

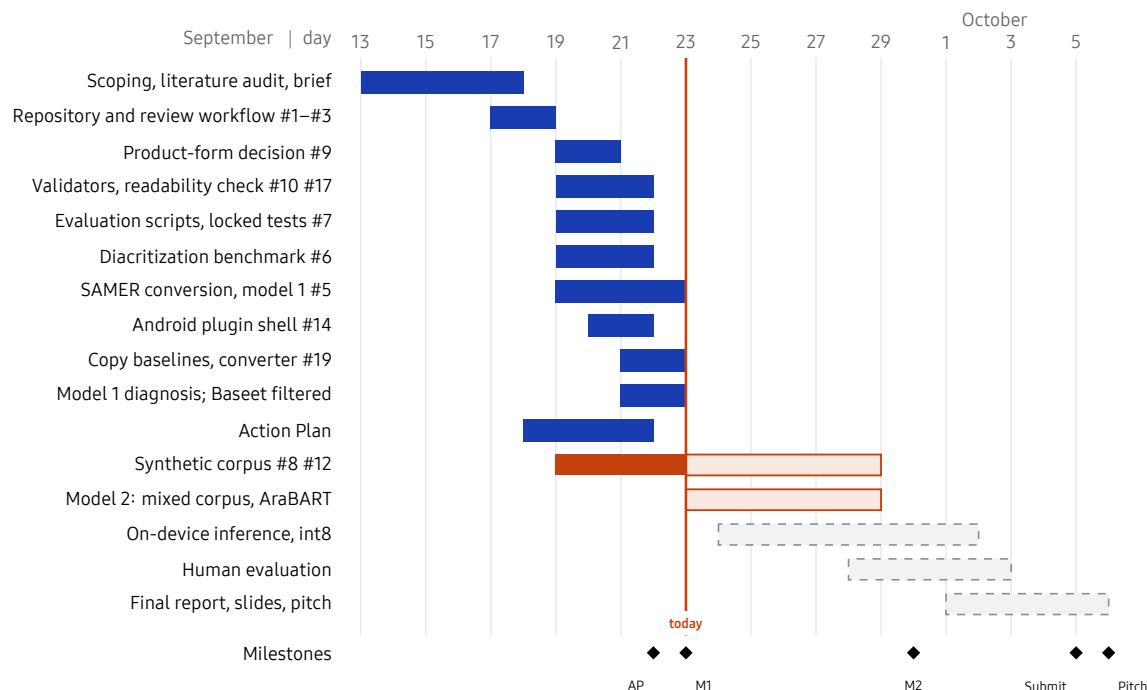


Figure 1. Project timeline as executed. Blue: done. Orange: in progress, solid to today. Dashed: not started. AP = Action Plan; M1, M2 = mentoring sessions. Numbers after a task are its GitHub issues.

Table 3. Corpora acquired. Sizes are rows in the files we hold.

Corpus	Size	Licence	Used for
SAMER [1]	14,343 / 2,983 / 3,277 pairs (train/dev/test)	research; written approval	model 1 training; locked test set
BAREC [2]	54,845 / 7,310 / 7,286 sentences	CC BY-SA 4.0	out-of-domain locked test set; readability labels
Baseet [3]	34,294 sources, three levels each; 31,104 after filtering	with permission	general-domain training pairs for model 2
DAASI [4]	1,509 pairs + 350 held out	CC0	human-written pairs; out-of-domain check
Tashkeela [5]	743 test sentences	open	diacritization benchmark

configuration file; parameter counts from its fp32 checkpoint size.

AraT5v2 went first because, on Baseet’s head-to-head evaluation (same split, same harness), it beat AraBART at all three simplification levels. AraBART is the challenger because it is 2.6 times smaller, which makes decoding on a phone cheaper. HPLT T5 is the reserve: it is the most recent, but needs custom loading code and has the shortest context.

Why the input is stripped of tashkeel. None of the three vocabularies was built for diacritized text. In AraT5v2 only 1.6% of pieces carry a diacritic, and a diacritized sentence costs 2.57 times as many tokens; AraBART cannot represent a diacritic at all and turns each one into an unknown token. The models learned Arabic from mostly undiacritized text and resolve ambiguous words from context, so a diacritic adds little they can use and a great deal they cannot. For every model trained on SAMER, which has no diacritics, the source is stripped before it reaches the simplifier; the diacritizer restores vowels afterwards, on the simplified text.

Model 1. We fine-tuned AraT5v2-base-1024 [6] (368M parameters), a text-to-text transformer

Table 4. Candidate base models. “Tashkeel” is measured on 1,000 diacritized BAREC dev sentences: vocabulary entries containing a diacritic, and how many more tokens the same sentences cost with diacritics than without.

	AraT5v2-base-1024	AraBART	HPLT T5 base (Arabic)
Family	T5 [9]: span-corruption pretraining, relative position buckets	BART [10]: denoising pretraining, learned positions	T5 in the NorT5 variant [11]
Layers (enc/dec), width	12/12, $d=768$, 12 heads, gated-GELU FFN 2048	6/6, $d=768$, 12 heads, GELU FFN 3072	12/12, $d=768$, 12 heads, FFN 2048
Parameters	368M (46% in two untied $110k \times 768$ embeddings)	139M	$\approx 294M$
Vocabulary	110,208 SentencePiece	50,002 SentencePiece BPE	32,768
Context	1,024 tokens	1,024 positions	512 positions
Pretraining	Arabic MSA and dialects; v2 of AraT5 [6, 8]	large Arabic corpora [7]	HPLT v3 web crawl, Arabic subset [12]
Tashkeel	1.6% of pieces; $2.57 \times$ tokens	0 pieces; every mark becomes <unk>; $4.53 \times$ tokens	0 pieces; $1.81 \times$ tokens
Licence	none stated on the model card	Apache-2.0	Apache-2.0
Status	model 1 trained	queued for model 2	held in reserve

pretrained on Arabic, to rewrite SAMER’s level-5 sentences into their level-3 simplifications. Training used AdamW (learning rate 10^{-4} , weight decay 0.01, 100 warm-up steps), an effective batch of 16, mixed precision and gradient checkpointing, for 10 epochs (3,250 steps) with seed 42. Pairs whose simplification is identical to the source make up 42% of SAMER train; they were capped at 20% of training so the model sees mostly real rewrites, and left untouched in dev and test. Each source was given the task prefix : بسط (*simplify*:). Every 200 steps SARI was computed on a fixed 500-row dev subset, and the checkpoint with the best SARI (step 3,000) was kept.

How it is evaluated. SARI [13] is the main metric, computed with the evaluate implementation, alongside BLEU, BERTScore [14] between source and output (meaning preservation), and the drop in predicted readability level. Every score is reported beside the *copy baseline* — the same metric for an output identical to the input — and split into rows a human changed and rows a human left unchanged (subsection 3.3 shows why the split matters). Model 1 was scored once on each locked test set after training was finished; no training or selection decision used those scores.

2.3. Workflow

Work runs through GitHub: each task is an issue with an owner, a reviewer and a definition of done; changes arrive as pull requests, are reviewed, and are squash-merged. By 23 September the team had opened 12 issues (9 closed) and merged 12 pull requests, with at least one from every member.

Figure 2 shows the data path. Two gates can reject data. The leakage check compares every training file against both locked test sets, by exact match and by character 5-gram Jaccard overlap of 0.8 or more; its first real use removed 9.3% of a published corpus (subsection 3.1). Generated pairs pass validators and human review before they may be trained on. The locked

test sets are fixed by row count and SHA-256 in a committed manifest, and both hashes were re-verified before the test runs reported here.

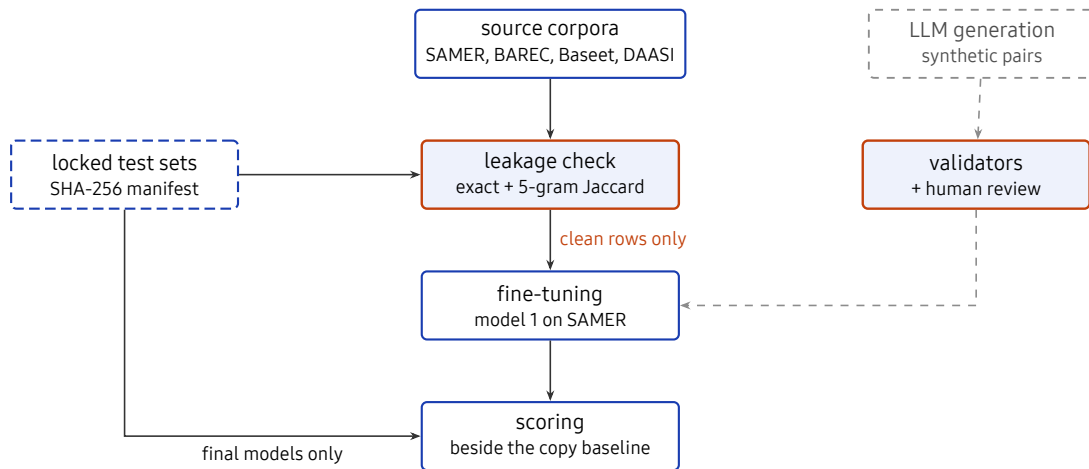


Figure 2. Data and evaluation workflow. Orange boxes are the gates that can reject data. Solid boxes are running; the dashed branch on the right (generated data) is built but has not yet delivered a corpus.

2.4. System Design

Figure 3 shows the system and the state of each part. The Android application registers an activity for `android.intent.action.PROCESS_TEXT` (mime type `text/plain`, Android 6.0 and later). A translucent theme lets the result sheet float over the host app, so the `SYSTEM_ALERT_WINDOW` overlay permission is never requested. Inference sits behind a `TextSimplifier` interface; today a mock engine implements it, so the interface, sheet and tests are complete while the model is integrated.

On the phone, models run through ONNX Runtime Mobile with greedy decoding and the tokenizer exported inside the model graph, so no Python runtime is needed. Diacritics are stripped before the simplifier and restored after it, for the reason measured in subsection 3.3. The simplifier’s size budget is 250 MB. AraT5v2 is 46% embedding matrices, so vocabulary pruning plus int8 quantization is estimated at about 223 MB; that is a calculation, and the on-device measurement is scheduled (subsection 4.2). The selected diacritizer is 4.8 MB, measured.

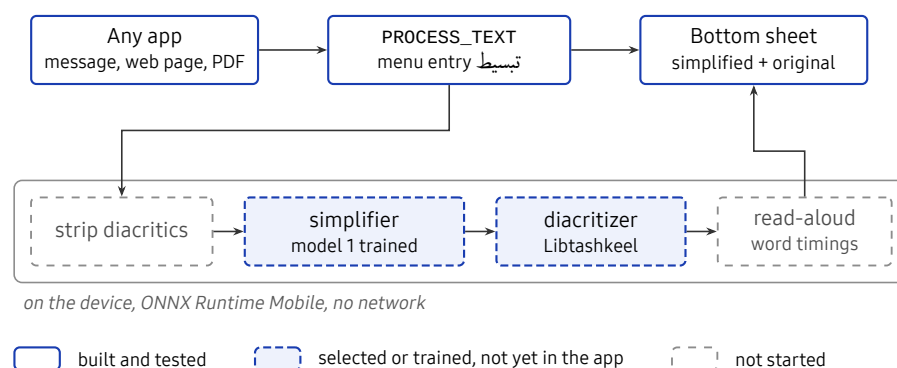


Figure 3. System architecture and build status on 23 September. The plugin is entered from the host app’s selection menu and answers with a sheet over it.

3. Results

3.1. Data Preprocessing

Conversion. SAMER's aligned levels were converted to source–target pairs (level 5 → level 3). A single converter (`make_eval_input.py`) turns each locked test file into the scoring format, keeps every row — including the 1,599 SAMER test rows (48.8%) where the two levels are identical — and checks the row count before writing.

Normalisation for comparison only. For leakage checks and scoring alignment, diacritics and punctuation are removed and alef, ؤ and ي/ي variants are unified. Training text keeps its original spelling.

Leakage checks. The official splits, model 1's training file and a sample of generated data were all checked against both locked test sets and found clean. Matches under five words are reported but not counted, because short formulaic fragments recur in any corpus; BAREC alone repeats 311 sentences between its own train and test splits.

Contamination found and removed. 3,190 of Baseet's 34,294 rows (9.3%) overlapped our locked test sets — 1,829 SAMER test sentences and 1,361 BAREC test sentences. Seen the other way, 47% of SAMER's substantive test sentences appear somewhere in Baseet. Training on it unfiltered would have inflated every later score without any visible error. The rows were removed and the remaining 31,104 re-checked: zero matches.

3.2. Exploratory Data Analysis (EDA)

We measured the corpora rather than assume them (Table 5, Figure 4); three differences turned out to explain model 1's behaviour.

Table 5. Corpus statistics, computed on the files we hold. Identity = target equals source after removing diacritics; length ratio = target words over source words, median.

Corpus	Rows	Median words	≤3 words	Diacritized	Identity	Length ratio
SAMER train	14,343	7	16.4%	0.0%	42.1%	1.00
SAMER test (locked)	3,277	6	23.6%	0.0%	48.8%	1.00
BAREC test (locked)	7,286	10	16.9%	53.8%	—	—
Baseet, strongest level	31,104	14	0.0%	36.5%	0.1%	0.73
DAASI	1,509	12	8.4%	32.3%	0.0%	1.00

- **SAMER has no diacritics at all**, while 54% of BAREC test and 36% of Baseet carry them. A model trained only on SAMER has never seen a diacritized input.
- **SAMER simplifies words, not sentences.** Its median length ratio is exactly 1.00: humans substituted vocabulary and kept structure. Baseet's strongest level shortens a 14-word median source to 9 words (ratio 0.73).
- **Half of SAMER needs no change.** 42–49% of pairs are identical, and SAMER splits by chapter rather than by novel, so its test set is fully in-domain. Both make SARI on SAMER read optimistically.

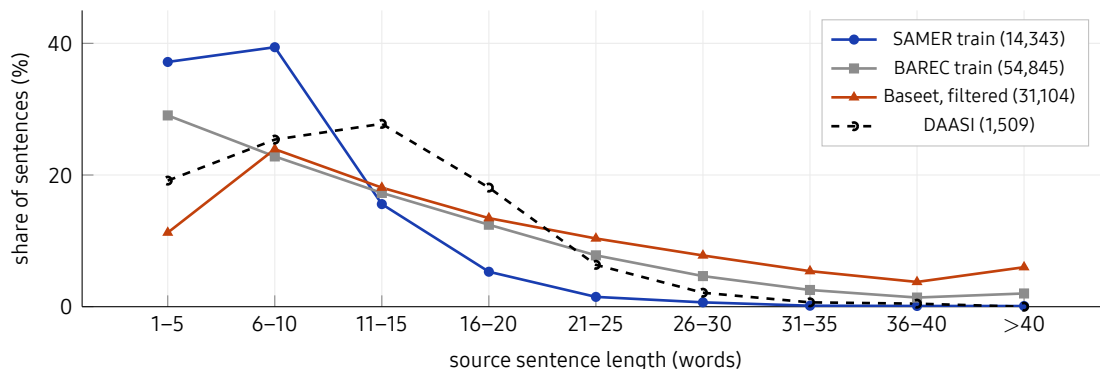


Figure 4. Source sentence length by corpus. SAMER is short literary text; Baseet and BAREC reach the long sentences a reader actually struggles with.

3.3. Modeling

The baseline comes first. Because half of SAMER needs no change, copying the input scores well: SARI 77.50 on the full test set, but only 56.06 on the 1,678 rows a human changed. The second number is the bar a model must clear. BAREC has no reference simplifications, so on it only the reference-free measures apply.

Model 1 on the locked test sets. We ran model 1 once on each locked test set, with the task prefix it was trained on, under greedy decoding (what the phone will run) and under beam search with four beams (Table 6, Figure 5). Before the test run the harness was validated on dev: it reproduced Ahmed’s 2,983 dev predictions exactly, row for row.

Table 6. Model 1 on test data with human references, scored once with `score.py`. SARI is split by whether the human reference changed the source (DAASI has no unchanged pairs). BERTScore compares output with source; readability drop is the fall in predicted BAREC level (positive = easier). DAASI sources and references have tashkeel stripped.

System	SARI all	SARI changed	SARI un-changed	BLEU	BERT-Score	Readability drop	Copy rate
SAMER test (3,277 rows, locked, in domain)							
Copy the input	77.50	56.06	100.00	72.03	1.00	0.00	100.0%
Model 1, greedy	75.88	61.83	90.62	71.68	0.98	0.08	65.3%
Model 1, beam 4	76.29	61.95	91.34	72.22	0.98	0.07	69.4%
DAASI held-out (350 pairs, government and insurance text, out of domain)							
Copy the input	45.05	45.05	—	25.25	1.00	0.00	100.0%
Model 1, greedy	43.87	43.87	—	23.66	0.99	-0.03	77.7%

Model 1 simplifies where it should, and over-edits where it should not. On the 1,678 test rows a human simplified, greedy decoding scores 61.83 against the copy baseline’s 56.06 (+5.77 SARI): the model is doing real work. On the 1,599 rows a human left alone it scores 90.62 against a perfect 100, because it edits 13.6% of sentences that needed no change. The two effects net to -1.62 overall. Beam search does not change the picture (61.95 on changed rows), so greedy decoding costs nothing on the phone.

It learned what SAMER teaches. Against the human references on test, the model returns 65.3% of rows unchanged where humans left 48.8%, and leaves 45.3% of the rows a human rewrote untouched. When it edits, 75.4% of its edits change a single word (humans: 49.5%), and its median output-to-input length ratio is 1.00: it substitutes vocabulary and never restructures

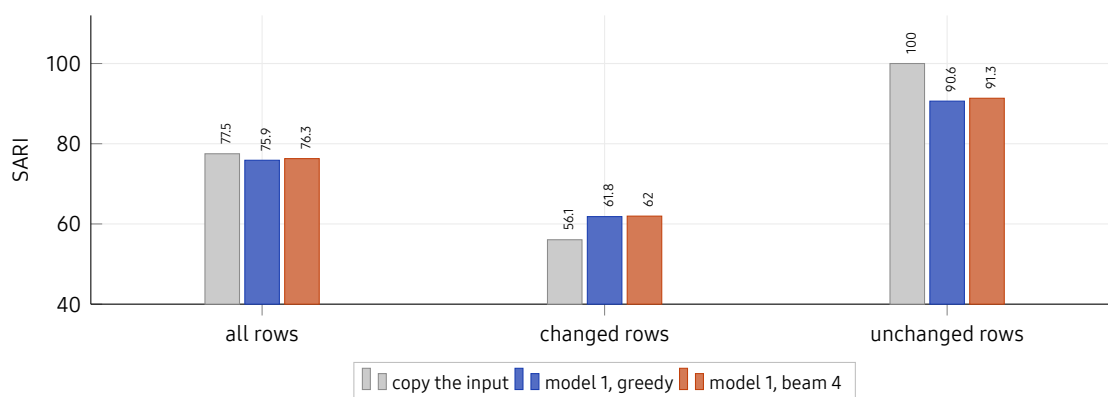


Figure 5. SARI on the locked SAMER test set by reference bucket. Model 1 clears the copy baseline where a human simplified, and loses to it where a human left the sentence alone.

(Figure 7a), exactly as the EDA predicts for SAMER. The readability classifier agrees: outputs are on average 0.08 of a level easier than their sources on BAREC’s 19-level scale, a change a reader would barely notice. Figure 6 shows how checkpoint selection contributed: copy rate was still falling at the last step, while the SARI that chose the checkpoint had been flat within noise since step 1,800, and validation loss had bottomed at step 2,200.

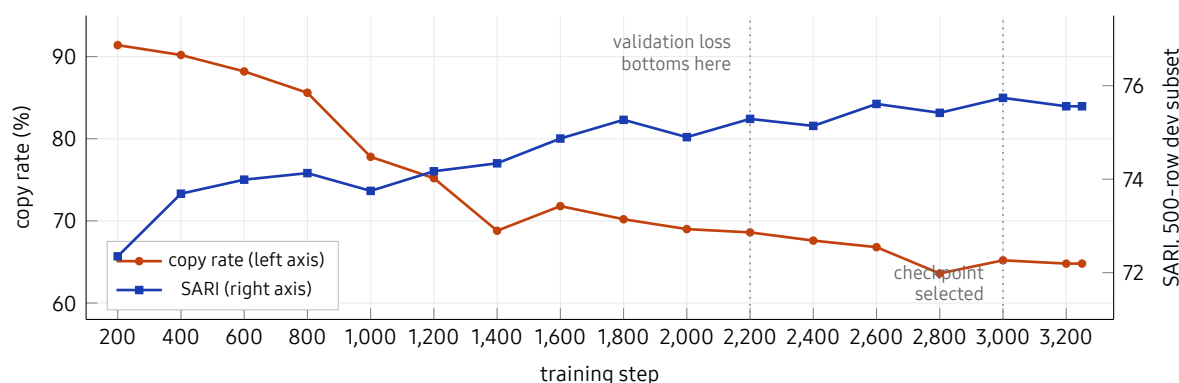


Figure 6. Model 1 training. The checkpoint was selected on a 0.18-point SARI difference while the model’s copying behaviour was still changing.

Out of domain, stripping tashkeel is what keeps it usable. BAREC’s 7,286 test sentences are general-domain text, and 3,918 of them carry tashkeel. We ran the model twice: on the raw text, and with tashkeel stripped from the source as the application does (Table 7, Figure 7b). On the sentences that carry tashkeel, raw input keeps only 63.7% of the source words and 31.9% of outputs lose more than half the sentence; stripped, the same sentences keep 94.6% and only 1.8% lose more than half. Stripping removes the model’s worst failure at no measurable cost.

Out of domain, it does not yet beat copying. DAASI’s held-out split is government and insurance text with human simplifications, so it gives an out-of-domain SARI. There model 1 scores 43.87 against a copy baseline of 45.05 (−1.19): it returns 77.7% of sentences unchanged, having learned from SAMER’s novels when *not* to edit, not how to simplify administrative prose. This is the gap model 2’s general-domain training data is chosen to close (Table 9).

Meaning errors are not rare. On dev, 832 of the model’s substitutions disagree with the human, 347 of them on words the human deliberately kept. Confirmed examples: عزلة (isolation) → استعباد (enslavement), ملبجم (bridled) → مسلح (armed). SARI and BLEU do not detect this class of error, which is why human error annotation is part of the evaluation.

Table 7. Model 1 on the locked BAREC test set (7,286 sentences, no reference simplifications), greedy decoding. Stripped: tashkeel removed from the source before the model sees it, as the application does.

Input	Copy rate	BERTScore	Readability drop	Source words kept	Rows losing > half
Stripped (as deployed)	68.6%	0.98	-0.04	94.9%	1.4%
Raw, with tashkeel	37.5%	0.91	-0.45	78.2%	17.5%

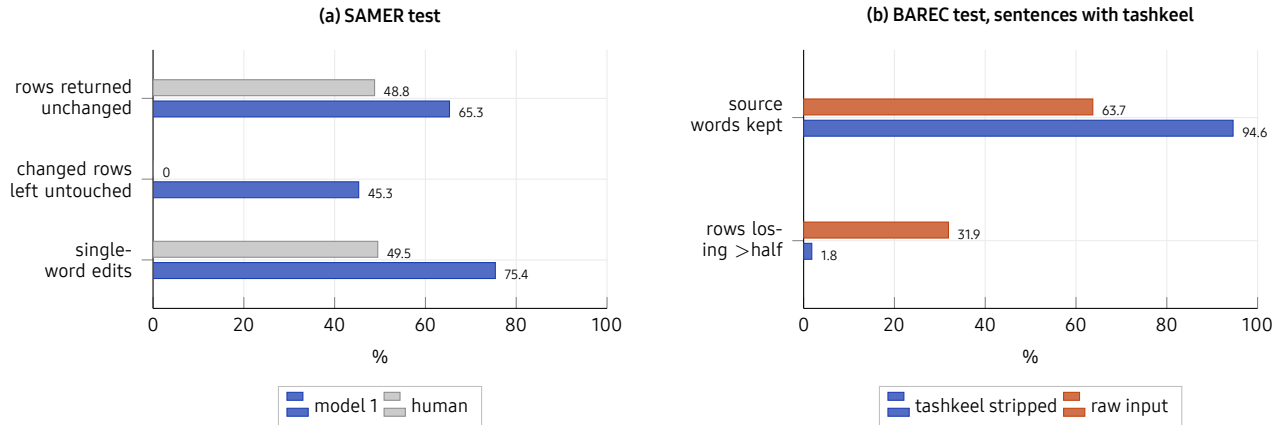


Figure 7. Model 1 behaviour. (a) SAMER test, greedy, against the human reference. (b) BAREC test, the 3,918 sentences that carry tashkeel, greedy, with and without tashkeel stripped from the input.

Diacritization. Four systems were benchmarked on the 743-sentence Tashkeela test fixture (Table 8). Libtashkeel was selected: CATT is statistically tied on accuracy (0.01 points, about six characters in 67,576) but 16 times larger and carries a non-commercial licence. CAMEL Tools was rejected for a stronger reason than accuracy: it changed the base letters of 32 of 100 sentences, corrupting the text it was asked to vocalise.

Table 8. Diacritization benchmark (#23). DER = diacritic error rate; base-letter corruption measured on 100 sentences.

System	DER	DER, no case	WER	Base letters changed	Size	Licence
CAMEL Tools MLE [15]	16.57%	13.97%	56.73%	32%	129 MB	MIT/GPL
CATT [16]	8.44%	6.92%	29.40%	0%	77.9 MB	CC BY-NC
Mishkal [17]	12.74%	10.12%	39.33%	0%	4.3 MB	GPLv3
Libtashkeel [18]	8.43%	6.91%	29.57%	0%	4.8 MB	MIT

3.4. User Interface

Figure 8 shows the interface as built. Selecting Arabic text in another app offers تبسيط; choosing it opens a Material bottom sheet over the app with the simplified text first, the original below it in a muted card, and controls for listening, copying and settings. Typography follows dyslexia guidance rather than density: 20 sp text, line height 1.6 plus 10 sp, letter spacing 0.04, and right-to-left text left *unjustified*, because justification in Arabic stretches words and opens uneven gaps. Swiping down, tapping outside or pressing Back closes the sheet and returns to the host app.

We documented the expected selection behaviour in five host apps — Chrome, WhatsApp, Keep, PDF readers and Gmail — including where تبسيط appears in each menu and which inputs are read-only. Confirming this on physical devices is part of the on-device milestone. Listen and

copy are visible but disabled until the read-aloud and model engines are connected.

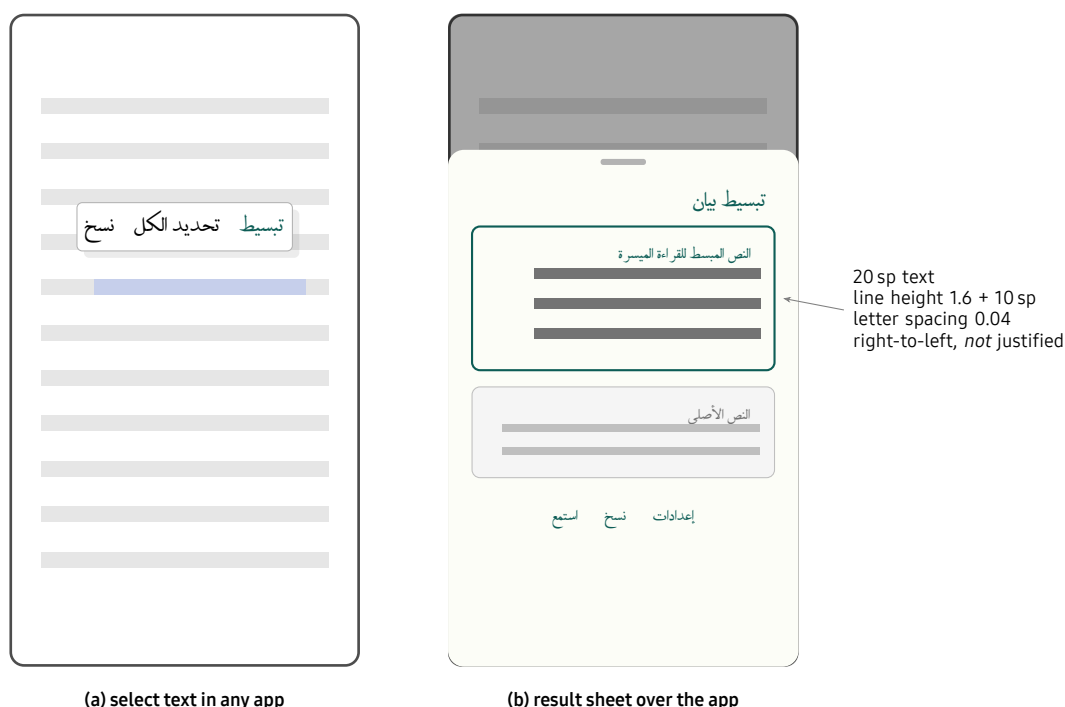


Figure 8. The interface as implemented (#21), drawn from its layout file. The sheet’s text is a placeholder: the engine behind it is still the mock.

3.5. Testing and Improvements

Testing. The application ships 41 unit and Robolectric tests covering intent extraction, empty input, view binding and the typography attributes. The leakage checker is tested against a planted duplicate and must exit non-zero. Running the scoring script end to end on real data, rather than fixtures, exposed a bug that put every BAREC row in the “unchanged” bucket; BAREC now has its own reference-free bucket. The same end-to-end run showed that the untrained AraT5v2 copies almost nothing and scores SARI 22.6 on 200 dev rows, so the gain from fine-tuning is real even though model 1 is conservative. Preparing the test run exposed a second harness bug: model 1 was trained with the input prefix :بسط, and the shared prediction script did not add it. Without the prefix the model inserts spurious words and scores 65.2 SARI on the test set, twelve points below copying; with it, the harness reproduces the training run’s predictions exactly.

Improvements. Each measured problem produced a change (Table 9).

4. Projected Impact

4.1. Accomplishments and Benefits

Built so far. A working Android plugin on the `PROCESS_TEXT` intent; a trained simplifier, scored on two locked test sets beside a copy baseline; a selected diacritizer with a four-system benchmark behind it; a hashed test-set manifest, a reusable leakage checker, and a scoring pipeline any member can run from one command.

Benefits. For readers: help at the moment and place the difficulty occurs, offline and private,

Table 9. What the measurements changed.

Finding	Change	Status
A copying model scores SARI 77.5 on SAMER test	Copy baseline and changed/unchanged split reported with every score	Done
9.3% of Baseet overlaps our test sets	Rows removed; leakage check required for every training file	Done
Model collapses on diacritized input	Strip diacritics before the simplifier; diacritized sources in training	Done / In progress
Checkpoint chosen on SARI noise, over copy-heavy dev	Stop on validation loss; select on changed-row SARI	In progress
Prediction script omitted the training prefix	Prefix made a required, recorded setting of every model	In progress
Model edits 13.6% of sentences that need no change	Identity rows kept in training at a controlled share; leave-unchanged mode as a level token	In progress
SAMER teaches word swaps only; below copying on DAASI (−1.19)	Model 2 corpus of ≈106,700 pairs: Baseet (three levels), SAMER, DAASI, identity rows capped at 4.6%	In progress
Published generation config stops at 20 tokens; licence tag says Apache-2.0	Fix max_length; set the non-commercial licence SAMER requires	Not started
One architecture tried	AraBART [7] (139M) trained on the same data as AraT5v2	Not started

Table 10. Objectives set in subsection 1.2, against the evidence on 23 September.

Objective	Evidence so far	Status
O1 Beat copying on changed rows, unseen data	Met in domain: SAMER test 61.83 vs 56.06 (+5.77). Not yet out of domain: DAASI 43.87 vs 45.05 (−1.19)	In progress
O2 Diacritize without changing letters	Libtashkeel: DER 6.91% without case endings, 0 of 100 sentences altered, 4.8 MB	Done
O3 Run on a phone within 250 MB	Plugin and sheet built and tested; model integration and measurement pending	In progress
O4 Evidence from readers	Protocol drafted; sessions not yet run	Not started

rather than in a separate app. For schools and teachers: a tool that works on the phones students already own. For the field: an evaluation protocol that exposes copy-heavy scores, and a documented case of test-set contamination in a public Arabic simplification corpus.

4.2. Future Improvements

Before submission (5 October). Train model 2 on the mixed corpus with the corrected selection rule, alongside AraBART, and choose on changed-row SARI; export the winner to ONNX, quantize it to int8 and measure size and latency on a real phone; connect diacritization and read-aloud; run the human evaluation with Arabic readers who have dyslexia; report annotator agreement on the generated data.

Known limitations. Automatic metrics miss meaning-altering substitutions, so human annotation stays necessary. SAMER’s test set is in-domain, so SARI on it overstates performance on news and forms; DAASI’s held-out split is our out-of-domain check.

Beyond the capstone. Sentence splitting, which SAMER does not teach and which matters most to readers with dyslexia; a diacritizer fine-tuned on simplified text; more domains in the corpus.

References

- [1] B. Alhafni, R. Hazim, J. D. Piñeros Liberato, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, pp. 16079–16093, 2024.
- [2] K. Elmadani, N. Habash and H. Taha. A Large and Balanced Corpus for Fine-grained Arabic Readability Assessment. *Findings of ACL*, pp. 16376–16400, 2025.
- [3] Baseet: a multi-level dataset for Arabic text simplification. <https://github.com/raya-y/Baseet>
- [4] DAASI: Arabic simplification pairs for government and insurance text. CC0.
- [5] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017.
- [6] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, pp. 628–647, 2022.
- [7] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, pp. 31–42, 2022.
- [8] A. Elmadany, E. M. B. Nagoudi and M. Abdul-Mageed. Octopus: A Multitask Model and Toolkit for Arabic Natural Language Generation. *ArabicNLP*, pp. 232–243, 2023. (Introduces AraT5v2.)
- [9] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [10] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *ACL*, pp. 7871–7880, 2020.
- [11] D. Samuel, A. Kutuzov, S. Touileb, E. Velldal, L. Øvrelid, E. Rønningstad, E. Sigdel and A. Palatkina. Nor-Bench – A Benchmark for Norwegian Language Models. *NoDaLiDa*, pp. 618–633, 2023.
- [12] L. Burchell et al. An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT). *ACL*, pp. 17452–17485, 2025.
- [13] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing Statistical Machine Translation for Text Simplification. *TACL*, 4:401–415, 2016.
- [14] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi. BERTScore: Evaluating Text Generation with BERT. *ICLR*, 2020.
- [15] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, pp. 7022–7032, 2020.
- [16] abjadai. CATT: Character-based Arabic Tashkeel Transformer. <https://github.com/abjadai/catt>
- [17] T. Zerrouki. Mishkal: Arabic text vocalization. <https://github.com/linuxscout/mishkal>
- [18] Libtashkeel, an ONNX Arabic diacritization model (MIT), via the `text2tashkeel` wrapper.

5. Team Member Review and Comment

<ATTACH A TEAM PICTURE HERE>

NAME	REVIEW and COMMENT
Marwan Elamami	
Abdulrahman Khengari	
Ahmed Alaeb	
Sanad Ali	
Mohammed Thabet	
Abdul Majid Mraied	

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	